

Mobile Messaging Application Database Specification

Overview

- Purpose: Define a local database schema for efficient storage, search, and retrieval of messages and related data in a mobile messaging app.
- Audience: Mobile engineers and backend developers implementing or maintaining the app database; secondary: technical architects, QA.
- Technology: SQLite with SQLDelight integration.

Prerequisites

- Familiarity with relational databases, SQLite, and SQLDelight.
- Understanding of messaging app concepts: users, contacts, messages, threads, statuses.

Schema Design

Users Table

- Stores user profiles.
- Key fields: user_id (PK), username, display_name, avatar_url, last_seen, status_message.

Contacts Table

- Stores contacts related to users.
- Fields: contact_id (PK), user_id (FK), contact_user_id (FK), nickname, blocked_status.

Threads/Conversations Table

- Represents message threads.
- Fields: thread_id (PK), thread_type (single/group), thread_name, created_at.

Messages Table

- Stores messages linked to threads.
- Fields: message_id (PK), thread_id (FK), sender_id (FK), content_text, content_type (text, image, video, etc.), sent_at, received_at, read_at.
- Support for multimedia: content_url or blob for media storage reference.

Message Status Table

- Tracks delivery and read status.
- Fields: status_id (PK), message_id (FK), status_type (sent, delivered, read), status_timestamp.

Constraints and Indexes

- Primary keys on all ID fields.
- Foreign keys enforce relationship integrity (user-contact, message-thread, message-sender).
- Indexes on thread_id and sender_id in Messages for fast retrieval.
- Index on content_text for search optimization (using FTS or LIKE-based indexes).

Example Queries

- Retrieve messages by thread ordered by sent_at.
- Search messages by keyword in content_text.

Integration with SQLDelight and SQLite

- Define schema in SQLDelight `.sq`` files.
- Use Kotlin-generated DAOs for type-safe queries.

Verification

- Schema supports efficient messaging and search.
- Referential integrity enforced.

Troubleshooting

- Missing indexes cause slow queries.
- Foreign key violations indicate data consistency issues.

References

- SQLite documentation
- SQLDelight GitHub repo

Summary

This specification details a SQLite database schema using SQLDelight for a mobile messaging app covering core data entities, constraints, and indexing strategies for efficient message handling and retrieval.